NYCU-EE IC LAB – SPRING 2026

Lab02 Exercise

Design: mini Image Signal Process(ISP)

Data Preparation

1. Extract test data from TA's directory:
   % tar -xvf ~iclabTA01/Lab02_2026spring.tar

Design Description

This project features the hardware implementation of a high-performance, area-efficient 12-bit Mini Image Signal Processor (ISP) pipeline. Designed to process 16 x 16 Bayer RAW input (RGGB pattern), the system performs real-time image reconstruction and enhancement through a series of specialized hardware modules. The pipeline consists of:

- **Black Level Correction (BLC)**
- **Lens Shading Correction (LSC)**
- **Directional 5×5 Defective Pixel Correction (DPC)**
- **3×3 Bilinear Demosaicing**
- **Color Correction Matrix (CCM)**

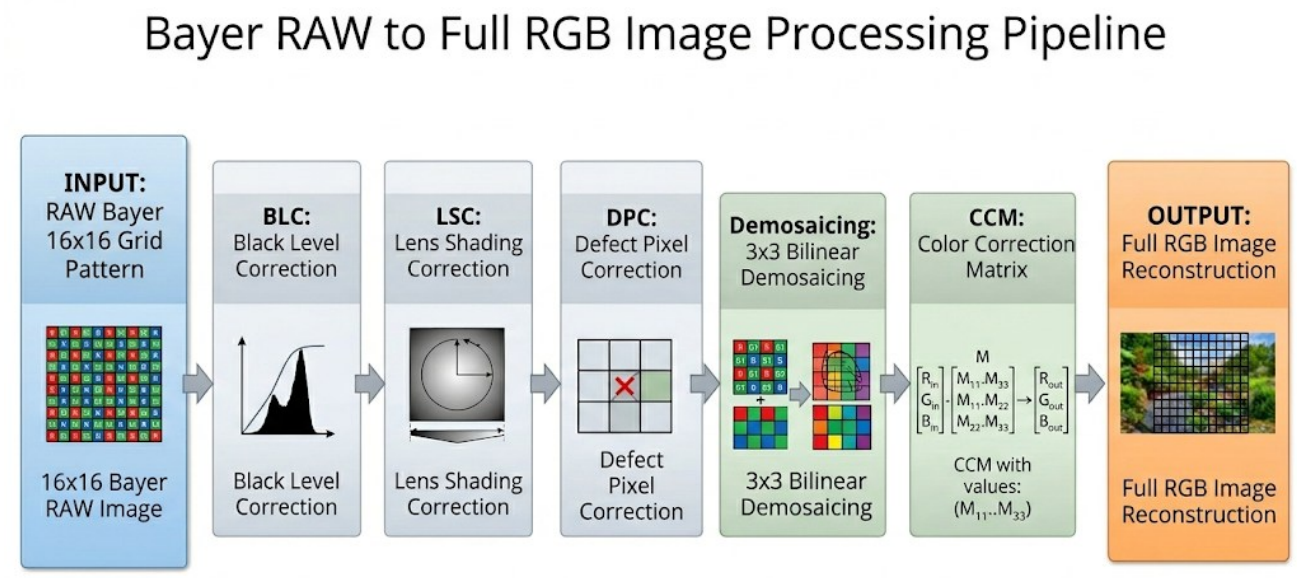


Fig. 1. Mini Image Signal Processor flow

Bayer Raw Image

A **Bayer RAW image** is the direct output from an image sensor before any image processing is applied. In a typical digital camera sensor, each pixel can only measure **one color component** because a **color filter array (CFA) is placed on top of the sensor.** The most commonly used CFA pattern is the **Bayer pattern**, which arranges the color filters in a repeating **RGGB pattern**.

As a result, each pixel location stores **only one color value (R, G, or B)** instead of full RGB information.

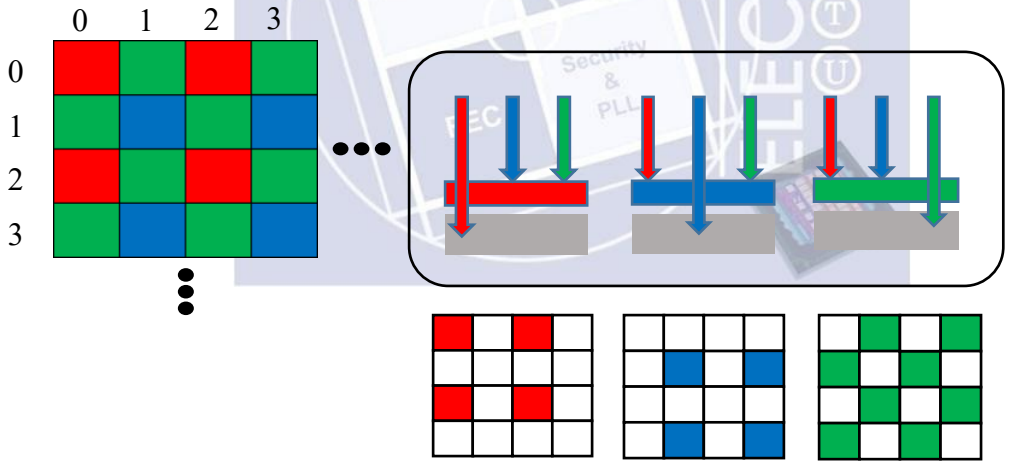


Fig. 2.Bayer Raw Image(RGGB)

The order of the input pixel(Row-major order):

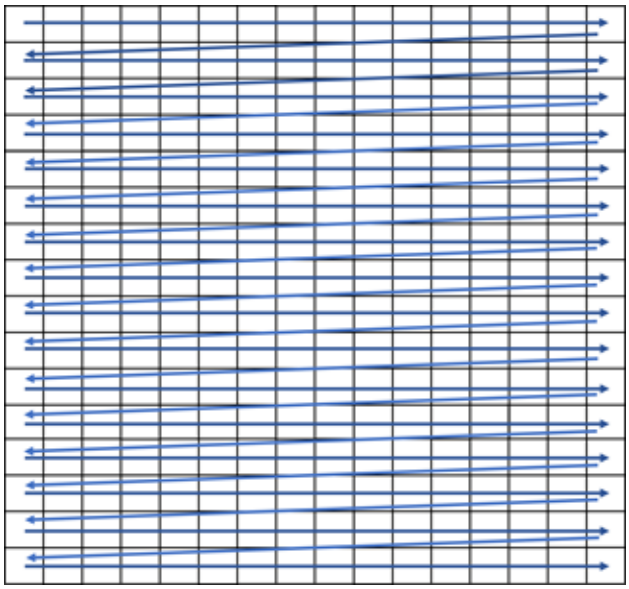


Fig. 3.The order of input pixel

Black Level Correction (BLC)

Sensor outputs include a constant black level offset due to analog circuitry.

The corrected pixel is computed as:

$$I_{\text{blc}} = \max(0, I_{\text{raw}} - B)$$

Where:

- I_{raw} is the 12-bit sensor input
- B is the programmable black level(RGGB)

In this design, each Bayer channel has an independent black level value:

Channel	Black Level
R (even row, even col)	64
Gr (even row, odd col)	48
Gb (odd row, even col)	52
B (odd row, odd col)	72

Table 1. Black Level of different channels

Lens Shading Correction (LSC)

Lens Shading Correction (LSC) is used to compensate for the vignetting effect, a physical phenomenon where the image brightness gradually decreases from the center toward the corners due to lens characteristics and sensor chief ray angle (CRA) mismatch.SC is performed in the RAW (Bayer) domain and must be completed before the Demosaic process to ensure color consistency.

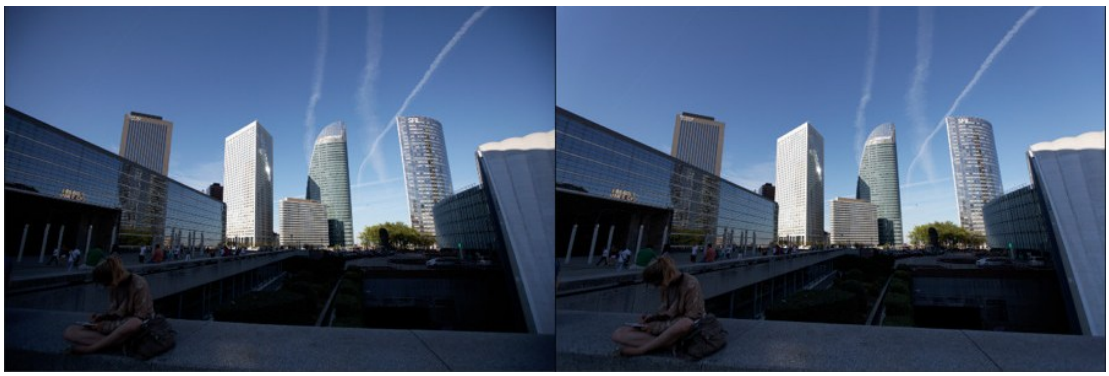
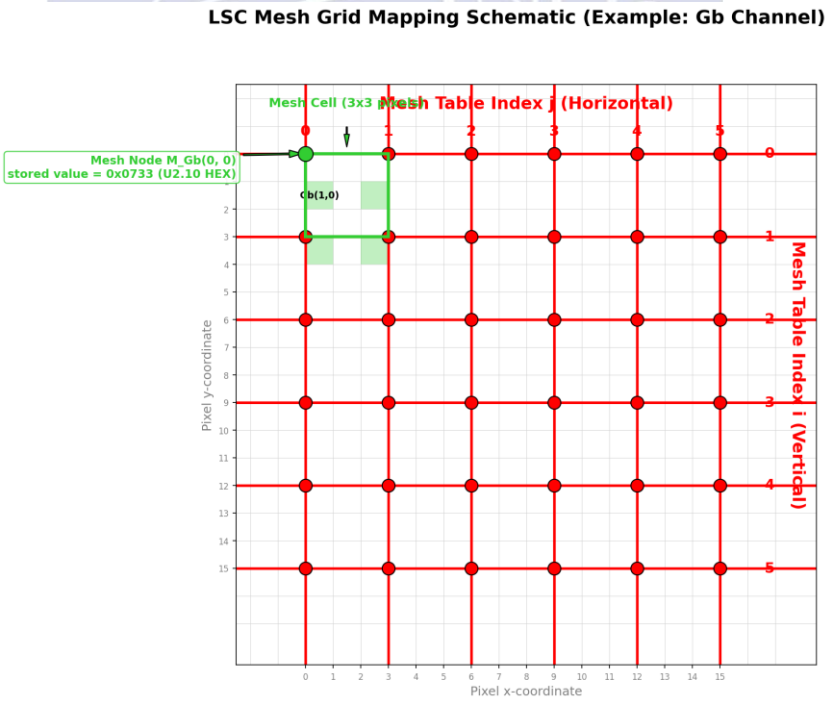


Fig. 4. Comparison of Lens Shading Correction

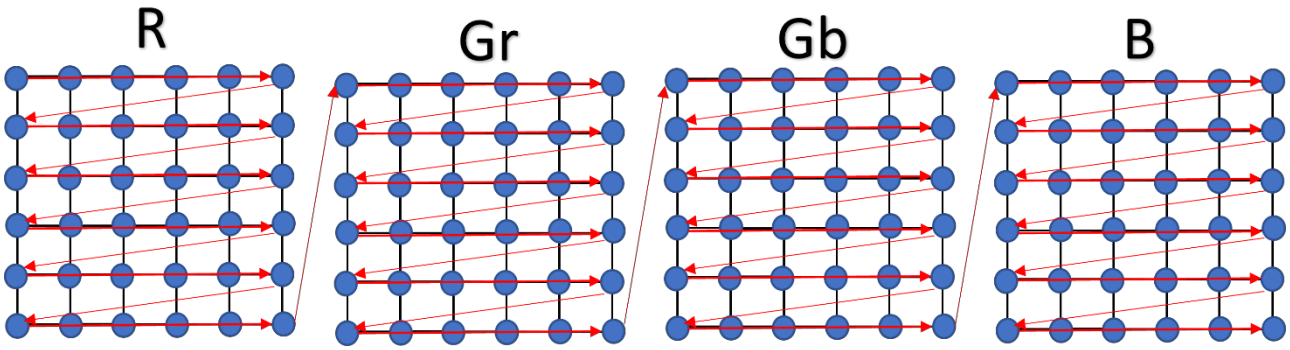
1. Gain Mesh Specification

To optimize hardware memory, the image is divided into a 5 x 5 grid of cells.

- Mesh Table: At the beginning, param_gain sequentially transmits the mesh gain tables for all four color channels.



The 144 mesh entries are streamed in via the param_gain input signal over 144 consecutive clock cycles, in the order R, then Gr, then Gb, then B, scanning row by row, left to right, top to bottom. Each color channel (R, G_r, G_b, B) has a 6 x 6 mesh gain table $M_c(i,j)$, where $i,j \in [0,5]$.



- Numerical Format: Mesh gains are stored in Fixed-point U2.10 format.
  - Example: A gain of 1.25x is represented as 1.25 x 1024 = 1280.
  - A gain would be 1.00x to 2.00x, meaning 1024~2048

2. Coordinate Mapping

For a pixel at coordinate (x, y), where x, y ∈ [0,15], the system identifies the surrounding mesh nodes and the relative position within the cell:

- Mesh Node Indices (x_0, y_0):

$$x_0 = min(\lfloor\frac{x}{3}\rfloor,4), y_0 = min(\lfloor\frac{y}{3}\rfloor,4)$$

The four active nodes are: $M(y_0, x_0), M(y_0, x_0 + 1), M(y_0 + 1, x_0), M(y_0 + 1, x_0 + 1)$.

$$\begin{aligned} g_{00} &= M_c(y_0, x_0) \\ g_{01} &= M_c(y_0, x_0 + 1) \\ g_{10} &= M_c(y_0 + 1, x_0) \\ g_{11} &= M_c(y_0 + 1, x_0 + 1) \end{aligned}$$

- Cell Fractional Position (r_x, r_y):

$$r_x = min(x - 3x_0, 2), r_y = min(y - 3y_0, 2)$$

3. Bilinear Interpolation Weights

The interpolation uses 8-bit fractional precision to compute the horizontal and vertical influence:

- Horizontal Weights:

$$d_x = \left\lfloor\frac{r_x \cdot 256 + 1}{3}\right\rfloor$$

$$i_x = 256 - d_x$$

- Vertical Weights:

$$d_y = \left\lfloor\frac{r_y \cdot 256 + 1}{3}\right\rfloor$$

$$i_y = 256 - d_y$$

4. Interpolation and Application

The gain G(x, y) is calculated by blending the four neighbor nodes $(g_{00}, g_{01}, g_{10}, g_{11})$:

$$G(x,y) = \frac{g_{00} \cdot i_x \cdot i_y + g_{10} \cdot i_x \cdot d_y + g_{01} \cdot d_x \cdot i_y + g_{11} \cdot d_x \cdot d_y + 2^{15}}{2^{16}}$$

The final corrected pixel P'(x, y) is calculated by applying the gain to the original RAW pixel P(x, y), followed by a 12-bit saturation clip:

$$P'(x,y) = min(\lfloor\frac{P(x,y) \cdot G(x,y) + 2^9}{2^{10}}\rfloor,4095)$$

Directional 5×5 Defective Pixel Correction (DPC)

Image sensors may contain defective pixels, which produce abnormal values that are much brighter (hot pixels) or much darker (dead pixels) than their surrounding pixels. These defects can be caused by manufacturing imperfections, aging of the sensor, or noise during image capture. If these defective pixels are not corrected, they will appear as visible artifacts such as bright or dark spots in the image, which significantly degrade image quality. Therefore, Defective Pixel Correction (DPC) is applied in the early stage of the ISP pipeline to detect and repair these abnormal pixels.

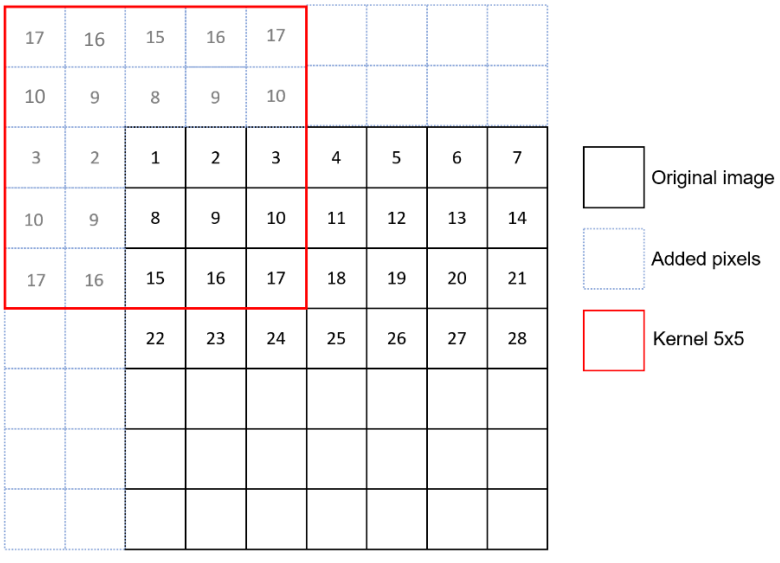


Fig. 5. Comparison of Defective Pixel Correction [1]

1. A 5×5 window is extracted.

   Supplemental: To handle boundary conditions and ensure the output image size remains W×H, reflect padding of 2 pixels is applied to the top, bottom, left, and right edges of the raw image. Let I(x,y) be the pixel at coordinate (x,y), where x in [0, W-1] and y in [0, H-1]. The padded pixel I'(x,y) is defined as:

$$I'(x,y) = \begin{cases} I(-x,y), & \text{if } x < 0 \text{ (Left boundary)} \\ I(2W-2-x,y), & \text{if } x \geq W \text{ (Right boundary)} \\ I(x,-y), & \text{if } y < 0 \text{ (Top boundary)} \\ I(x,2H-2-y), & \text{if } y \geq H \text{ (Bottom boundary)} \end{cases}$$



2. Four directional vectors are formed:

   Four directional vectors are formed from the 5×5 window:

   - **Horizontal (H):** $[H_0, H_1, H_2, H_3]$
   - **Vertical (V):** $[V_0, V_1, V_2, V_3]$
   - **Diagonal 1 (D1):** $[D_{10}, D_{11}, D_{12}, D_{13}]$
   - **Diagonal 2 (D2):** $[D_{20}, D_{21}, D_{22}, D_{23}]$

D_{10}		V_0		D_{23}
	D_{11}	V_1	D_{22}	
H_0	H_1		H_2	H_3
	D_{21}	V_2	D_{12}	
D_{20}		V_3		D_{13}

3. For each of the four orientations (Horizontal, Vertical, and two Diagonals), a **Smoothness Score** is dynamically computed to evaluate the local texture consistency. This is achieved by calculating the **Sum of Absolute Differences (SAD)** between the four neighboring pixels in that direction and the **corresponding median** of the four neighboring pixels:

$$Score_{dir} = \sum_{i=0}^{3} |Pixel_{dir,i} - Median_{dir}|$$

4. The direction with the minimum SAD is selected, and its **median** is designated as the Target. The center pixel **P** is replaced by **Target** if:

$$|P - \text{target}| > \text{TH}$$

Where:

- **P** is the center pixel
- **Target** is median of the direction with the minimum SAD
- **TH** is 320

Tie-Breaking Rule for Equal SAD Scores

If two or more directions produce the same minimum SAD score, the direction is selected according to the following fixed priority order:

Horizontal (H) > Vertical (V) > Diagonal 1 (D1) > Diagonal 2 (D2)

For example, if H and V both have the minimum SAD, H is chosen.

3×3 Bilinear Demosaicing

Each pixel location contains only one color component (R, G, or B).The missing two color components must be reconstructed using neighboring pixels.

- **Interpolation Rules**

1. Boundary Handling & Window Extraction
Let the center pixel be located at coordinate (x, y). To compute the missing color components, we define its 8-neighborhood within a 3×3 window as follows:

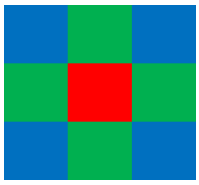
$$\begin{matrix} NW & N & NE \\ W & C & E \\ SW & S & SE \end{matrix}$$

To maintain the original image resolution of W×H, reflect padding of 1 pixel is applied to all four edges. For each pixel P(x,y), a 3×3 window is extracted. If a neighbor's coordinate (x', y') falls outside the image boundaries, the pixel value is mirrored as follows:

- If x' < 0, x' = |x'|
- If x' ≥ W, x' = 2W - 2 - x'
- (The same logic applies to the y coordinate)

(A) At R Location

At an **R pixel** position, the Red component is original, while the **Green** and **Blue** components are missing and must be interpolated from the 3×3 neighborhood.



1. Green Interpolation

The missing Green value is calculated by averaging the four direct neighbors (North, South, East, West):

$$G = \frac{N + S + E + W}{4}$$

2. Blue Interpolation

The missing Blue value is calculated by averaging the four diagonal neighbors (North-West, North-East, South-West, South-East):

$$B = \frac{NW + NE + SW + SE}{4}$$

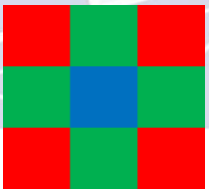
3. Final RGB Output

The final triplet for the pixel located at the Red sensor site is:

$$\begin{cases} R_{\text{out}} = R_{\text{center}} \\ G_{\text{out}} = \dfrac{N+S+E+W}{4} \\ B_{\text{out}} = \dfrac{NW+NE+SW+SE}{4} \end{cases}$$

(B) At B Location

At a **B pixel** position, the Blue component is original, while the **Red** and **Green** components are missing and must be interpolated from the 3×3 neighborhood.



1. Green Interpolation

The missing Green value is calculated by averaging the four direct neighbors (North, South, East, West):

$$G = \frac{N+S+E+W}{4}$$

2. Red Interpolation

The missing Red value is calculated by averaging the four diagonal neighbors (North-West, North-East, South-West, South-East):

$$R = \frac{NW+NE+SW+SE}{4}$$

3. Final RGB Output

The final triplet for the pixel located at the Blue sensor site is:

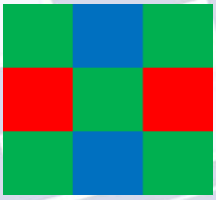
$$\begin{cases} R_{\text{out}} = \dfrac{NW+NE+SW+SE}{4} \\ G_{\text{out}} = \dfrac{N+S+E+W}{4} \\ B_{\text{out}} = B_{\text{center}} \end{cases}$$

(C) At G Location

The interpolation at a **Green pixel** depends on its position within the Bayer grid. There are two distinct cases based on whether the Green pixel is located on a **Red row** or a **Blue row**.

1. G on R Row (G in R-G-R)

At this location, the Green value is original. The horizontal neighbors are **Red**, and the vertical neighbors are **Blue**.



- **Red Interpolation (Horizontal):**

$$R = \frac{W + E}{2}$$

- **Blue Interpolation (Vertical):**

$$B = \frac{N + S}{2}$$

Final Output:

$$\begin{cases} R_{\text{out}} = \dfrac{W + E}{2} \\ G_{\text{out}} = G_{\text{center}} \\ B_{\text{out}} = \dfrac{N + S}{2} \end{cases}$$

---

2. G on B Row (G in B-G-B)

At this location, the Green value is original. The horizontal neighbors are **Blue**, and the vertical neighbors are **Red**.



- **Red Interpolation (Vertical):**

$$R = \frac{N + S}{2}$$

- **Blue Interpolation (Horizontal):**

$$B = \frac{W + E}{2}$$

Final Output:

$$\begin{cases} R_{\text{out}} = \dfrac{N+S}{2} \\ G_{\text{out}} = G_{\text{center}} \\ B_{\text{out}} = \dfrac{W+E}{2} \end{cases}$$

Color Correction Matrix (CCM)

Even after Demosaicing provides a full RGB image, the colors are not yet accurate to human perception. This is due to several physical factors:

- **Spectral Cross-talk:** The Red, Green, and Blue micro-filters on a CMOS sensor are not "perfect." For example, a Red filter might still allow some Green light to pass through. This results in "muddy" or desaturated colors.
- **Sensor Response vs. Human Eye:** The sensitivity of the silicon sensor (CFA) differs significantly from the Human Visual System (HVS).
- **Color Space Alignment:** The RAW RGB values are in the "Sensor RGB" space. CCM is required to transform these values into a standard color space (such as sRGB) to ensure that the red looks "red" and the skin tones look natural.

Mathematically, this is achieved by a 3x3 linear transformation where each output color is a weighted sum of all three input colors, allowing the ISP to "subtract" the overlapping spectral noise.

1. Matrix Linear Combination

The Color Correction Matrix (CCM) converts the input RGB values into corrected RGB values using a 3×3 matrix multiplication.

For each pixel, the input 12-bit RGB values are combined linearly:

$$\begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

The floating-point equivalent CCM used in this design is approximately:

$$\mathbf{C} \approx \begin{bmatrix} 1.074 & -0.049 & -0.049 \\ -0.049 & 1.074 & -0.049 \\ -0.049 & -0.049 & 1.074 \end{bmatrix}$$

This matrix slightly increases the target channel while subtracting small contributions from the other channels to improve color accuracy.

2. Fixed-Point Hardware Implementation

In hardware, floating-point numbers are inefficient. Therefore, we use fixed-point integer coefficients. All coefficients are scaled by 2^{10} :

$$\mathbf{C}_{int} = \begin{bmatrix} 1100 & -50 & -50 \\ -50 & 1100 & -50 \\ -50 & -50 & 1100 \end{bmatrix}$$

Before bit-shifting, we add 2^9. Mathematically, this is equivalent to adding 0.5 in the decimal domain to achieve correct rounding to the nearest integer:

$$R_{round} = (C_{int} \cdot R_{raw} + 512)$$

The result is right-shifted by 10 bits to return the values to the original 12-bit dynamic range

$$R_{shift} = R_{rounded} >> 10$$

To prevent data overflow or underflow, the values are constrained to the legal 12-bit range (0 to 4095):

$$R' = max(0, min(R_{shifted}, 4095)$$

I/O specification

Signals name	Direction	Bit Width	Definition
clk	input	1	Clock.
rst_n	input	1	Asynchronous active-low reset.
in_valid	input	1	High when input signal is valid.
in	input	12	12-bit RAW Bayer pixel input data (RGGB format). One pixel per cycle.
param_valid	input	1	High when param_gain signal is valid.
param_gain	input	12	12-bit gain 6×6 mesh table. Channel order is R → Gr → Gb → B. Scan order is row-major, left-to-right, top-to-bottom. The entire 144 entries (four 6×6 tables) are streamed continuously with no gaps.
out_valid	output	1	High when output is valid.
r_out	output	12	12-bit red channel output after processing.
g_out	output	12	12-bit green channel output after processing.
b_out	output	12	12-bit blue channel output after processing.

Specifications

1. Top module name：ISP (Filename: ISP.v)

Reset

2. It is an asynchronous reset and active-low architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals.
3. The reset duration must not exceed three clock cycles.

Param_gain signal

4. param_gain is asserted only once at the beginning. After that, the same set of parameters is used for all subsequent operations.
5. Scan order is row-major, left-to-right, top-to-bottom.

6. The four tables (R → Gr → Gb → B) are streamed continuously over 144 consecutive cycles without interruption.
7. After the param_gain signal ends, the input signal will be driven immediately, meaning no cycle gap between the input signal.

Input and Output signal

8. The next group of inputs will come in 1~3 cycles after your out_valid pull down.
9. The out_valid cannot overlap in_valid.
10. The latency of your design in each pattern should not be larger than 1500 cycles. The latency is the clock cycles between the falling edge of the **in_valid** and the falling edge of the **out_valid**.
11. For every cycle, the output must simultaneously produce all three color values (R, G, and B).
12. The input and output data are arranged in row-major order.
13. When out_valid is low, the r_out, b_out, and g_out should be zero.

Synthesis

14. The maximum clock period is set to 20 ns, and you can determine the clock period by yourself.
15. The synthesis result of data type cannot include any LATCH.
16. After synthesis, you can check ISP.area and ISP.timing in the folder "Report".
17. The total cell area should not larger than 2,500,000 μm2.
18. The slack in the timing report should be non-negative and the result should be MET.

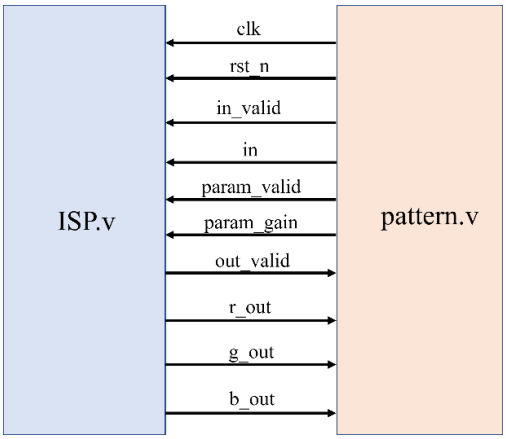
Gate level

19. The gate level simulation cannot include any timing violation.

else

20. Any words with "error", "FAIL", "latch" or "Congratulation" can't be used as variable name.
21. Ten test cases will be provided, but hidden test cases will be used during the demo.

Block Diagram



Note

1. **Grading policy:**
   RTL and gate-level simulation correctness: 70%
   Performance ($Area^2$ *Cycle Time*Execution Cycle): 30%

2. **Please submit your design through Lab02_2026spring/09_SUBMIT/**
   - 1st_demo deadline: 2026/3/16(Mon.) 12:00:00
   - 2nd_demo deadline: 2026/3/18(Wed.) 12:00:00 (Score = 50pts)
   - If uploaded files violate the naming rule, you will get 5 deduct points.

3. Template folders and reference commands:

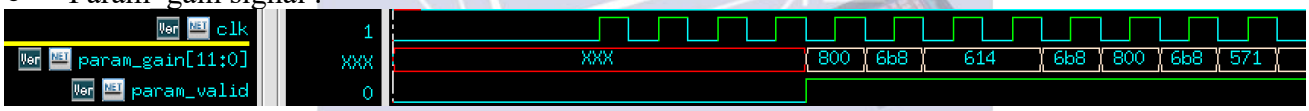
01_RTL/	(RTL simulation)	**./01_run_vcs_rtl**
02_SYN/	(Synthesis)	**./01_run_dc_shell**

(Check the design if there's latch or not in ***syn.log***)
(Check the design's timing in /Report/ ISP.***timing***)

03_GATE /	(Gate-level simulation)	**./01_run_vcs_gate**
09_SUBMIT/(tar all your design)		**./00_tar**
09_SUBMIT/(submit files)		**./01_submit**
09_SUBMIT/(check files)		**./02_check**

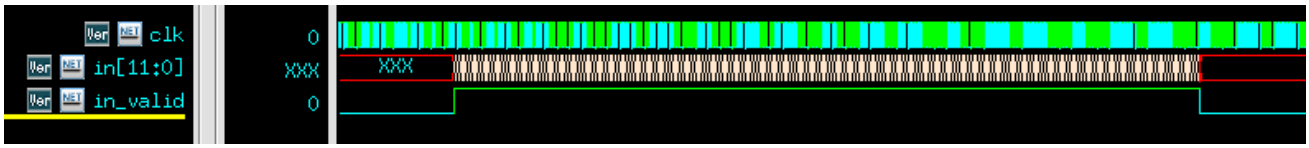
Sample Waveform

- Param_gain signal :

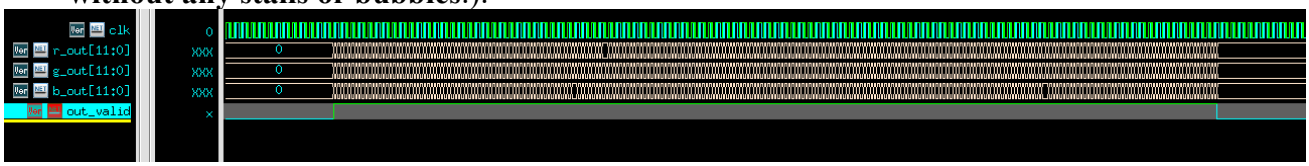


- Input signal (A fram is consecutive 256 cycles):

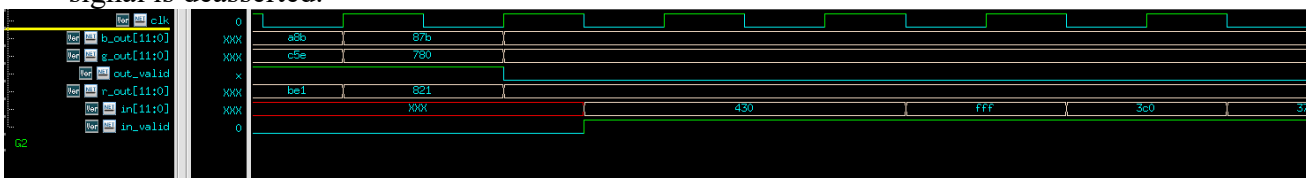




- Output signal(The output signals r_out, g_out, and b_out must operate like a **continuous stream**; once the first pixel is asserted, the pipeline must provide **256 consecutive cycles of valid data without any stalls or bubbles**.):



- The input data for the subsequent frame must be streamed 1 to 3 clock cycles after the out_valid signal is deasserted.



Reference

[1] C.-Y. Cho, W.-S. Wang, T.-M. Chen, and C.-N. Liu, "Real-Time Photo Sensor Dead Pixel Detection for Embedded Devices," in *Proceedings of the Industrial Technology Research Institute (ITRI)*, Hsinchu, Taiwan.